

COMPTE RENDU TECHNIQUE

WARAQ

Pipeline de Classification Au- tomatique et de Découpage

Plateforme d'Intelligence pour Appels d'Offres

Réalisé par :

Achraf Ait Lahcen

Date : 30 juillet 2026

Année Universitaire : 2025 – 2026

1. Contexte & Objectif du Système

Dans le cadre de la plateforme **Waraq**, le volume important de documents d'appels d'offres (*tenders*) extraits par scraping nécessite une organisation et une catégorisation rigoureuses. Le service de classification post-scraping a pour rôle de :

1. **Regrouper et traiter** automatiquement les documents par dossier d'Appel d'Offres.
2. **Déterminer le type précis** de chaque document (ex : CPS, Règlement de Consultation, Avis, etc.).
3. **Identifier et découper** les fichiers PDF combinés (fichiers contenant plusieurs sous-documents) en sous-segments distincts.
4. **Classer physiquement** les fichiers dans une arborescence chronologique structurée sur le disque.
5. **Consigner l'intégralité du processus** dans la base de données via une traçabilité d'audit détaillée (`ClassificationAuditLog`).

2. Architecture de la Base de Données (Modèles)

Le pipeline s'appuie sur une relation à trois niveaux implémentée avec **SQLAlchemy** :

- **Tender (Dossier d'Appel d'Offres)** : Représente le dossier parent qui regroupe un ensemble de pièces jointes.
- **TenderDocument (Document Physique)** : Stocke le statut du fichier (`is_classified`), son chemin source et destination (`classified_file_path`), son type identifié (`file_type`), ainsi que les métadonnées d'analyse (`analysis_metadata`).
- **ClassificationAuditLog (Journal d'Audit)** : Assure la traçabilité complète de l'apprentissage et des décisions IA/règles. Il conserve le score de confiance, le type prédit (`predicted_type`), le modèle utilisé (`model_used`), la justification (`classification_reason`), et le statut de validation par un opérateur humain (`validation_status="PENDING"`).

3. Déroulement Détaillé du Pipeline

Le traitement s'exécute selon une séquence algorithmique à 4 étapes majeures :



3.1. Étape 1 : Regroupement et Chargement du Contextual Few-Shot

- **Optimisation de la sélection** : Le pipeline cible uniquement les dossiers Tender possédant au moins un document non classifié (`is_classified == False`).
- **Moteur d'apprentissage (WaraqLearningEngine)** : Chargement des exemples de corrections humaines passées (`contexte_few_shot`) afin de nourrir le prompt de l'IA et d'améliorer la précision sur les cas ambigus.

3.2. Étape 2 : Classification Hybride (Règles Primitives + Fallback IA)

Pour chaque document d'un dossier :

1. **Analyse Primitive** : Vérification rapide basée sur des règles métier applicables au nom du fichier et à son extension (`appliquer_types_primitifs`).
2. **Validation LLM** : Même lorsqu'une règle primitive s'applique, un texte condensé (extrait des 10 premières pages) est envoyé au LLM pour confirmer le type (`verifier_ou_classifier_par_llm`).
3. **Fallback IA** : Si la règle primitive est absente ou invalidée par le LLM, le pipeline passe sur une analyse de contenu approfondie (`determiner_type_par_ia`) en injectant les exemples *few-shot*.

3.3. Étape 3 : Inspection et Découpage Automatique des PDF

Lorsqu'un fichier `.pdf` contient plusieurs pièces rassemblées en un seul document (ex : un CPS suivi de l'Avis de publication) :

- La fonction `verifier_et_decouper_pdf` segmente le fichier source en plusieurs sous-fichiers distincts.
- Chaque segment reçoit son propre type propre nettoyé (`t_final_clean`).

3.4. Étape 4 : Organisation Physique et Traçabilité BDD

- **Arborescence Chronologique sur Disque** : Le fichier est déplacé dans un répertoire structuré selon la date du dossier et son type :
`classified/YYYY/MM/DD/{Reference_Heure}/{Type_Document}/{Nom_Fichier}`
- **Entrée Principale (Document d'origine)** : Mise à jour des champs `is_classified=True`, `file_type`, `classified_file_path` et `classification_reason`.
- **Gestion des Segments Découpés** : Pour chaque sous-segment issu d'un découpage PDF (à partir de l'index 1), une nouvelle entrée `TenderDocument` est créée dynamiquement dans la base de données.
- **Création systématique de l'Audit Log** : Chaque entrée (document principal ou sous-segment) enregistre une instance de `ClassificationAuditLog` pour permettre l'amélioration continue et la validation humaine future.

4. Stratégie de Résilience & Gestion des Transactions (BDD)

Le code met en place une gestion stricte du cycle de vie des sessions **SQLAlchemy** :

Gestion des Transactions

- **Isolation par Document** : Le traitement de chaque document est encapsulé dans un bloc `try / except` individuel.
- **Rollback Cible** (`db.rollback()`) : En cas d'erreur sur un fichier (fichier manquant, échec de découpage, incohérence de champ), la transaction du document échoué est annulée sans bloquer le reste du pipeline ni polluer le dossier.
- **Flushing explicite** (`db.flush()`) : Utilisé lors du découpage PDF pour générer l'identifiant unique (UUID) du nouveau morceau avant de créer son `ClassificationAuditLog` lié.
- **Fermeture Garantie** (`finally: db.close()`) : Assure la libération des connexions au pool de la base de données PostgreSQL à la fin du traitement global.

5. Résumé des Modifications Récentes (Correction du Bug)

Résolution du Crash lors de l'Instanciation de l'Audit Log

Problème résolu : Une tentative d'affectation de l'argument `classification_description` lors de la création de `ClassificationAuditLog` provoquait un crash (*invalid keyword argument*).

Correction appliquée : Alignement strict avec le modèle SQLAlchemy en fusionnant la description d'analyse directement dans le champ `classification_reason` et en supprimant l'argument obsolète lors des instanciations du log.